

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
GitHub Repository	https://github.com/Dharika-2006/ML-LAB-ASSIGNMENTS
Student Name	Dharika Shree K
Register Number	3122247001014
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30.08.26

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XG-Boost, and Stacking – on the Wisconsin Diagnostic Breast Cancer dataset. Each model is trained and validated both without PCA (original 30-feature space) and with PCA (reduced feature space retaining 95% variance), with hyperparameter tuning and 5-fold cross-validation performed under both settings, in order to compare accuracy, F1-score, and fold-wise stability.

2 Problem Statement

Problem: Classify breast cancer tumors as malignant or benign using ten different classifiers, and determine whether reducing the feature space via PCA improves or degrades classification performance and cross-validation stability compared to using the original feature space.

Input: The Wisconsin Diagnostic Breast Cancer dataset, containing 569 samples with 30 numerical features describing characteristics of breast cell nuclei, with each sample labelled malignant or benign.

Output: Tuned classifiers under both No-PCA and With-PCA settings, fold-wise and average 5-fold cross-validation accuracy/F1-score, test-set accuracy and F1-score, confusion matrices and ROC curves for the best model in each setting, and a final comparison identifying which models benefited from PCA and which did not.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	Scikit-learn (<code>load_breast_cancer</code>)
Number of Samples	569
Number of Features	30 (original); 10 (after PCA, retaining $\geq 95\%$ variance)
Number of Classes	2 (malignant = 212 samples, benign = 357 samples)
Missing Values	None (0 missing values)

Train-Test Split	80% training (455 samples) / 20% testing (114 samples), stratified, <code>random_state=42</code>
------------------	--

4 Exploratory Data Analysis

- Dataset overview
- Class distribution
- PCA scree plot
- PCA cumulative explained variance

Figure: Class Distribution

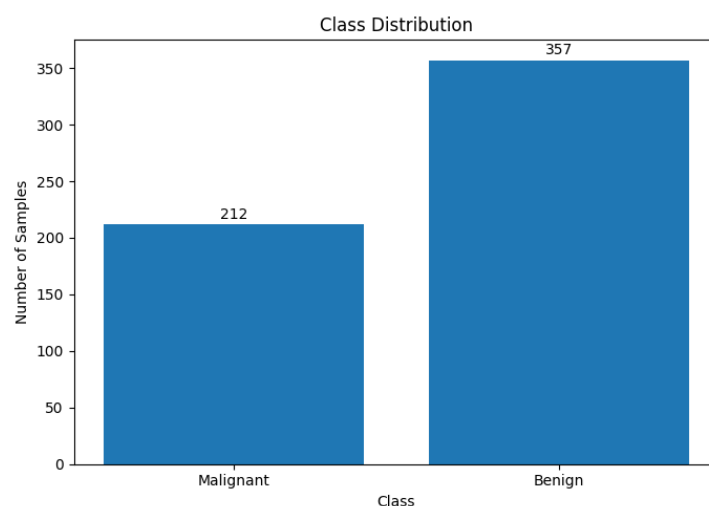


Figure 1: Class distribution of malignant and benign samples

Inference: The dataset contains 357 benign samples and 212 malignant samples, a moderate class imbalance of roughly 63:37. Stratified splitting was therefore used for both the train-test split and the 5-fold cross-validation folds to preserve this ratio. Because of this imbalance, F1-score was tracked alongside accuracy for every model in both the No-PCA and With-PCA settings, rather than relying on accuracy alone. This distribution is consistent across all ten models compared in this experiment, since the same train-test split and folds were reused throughout.

Figure: Scree Plot

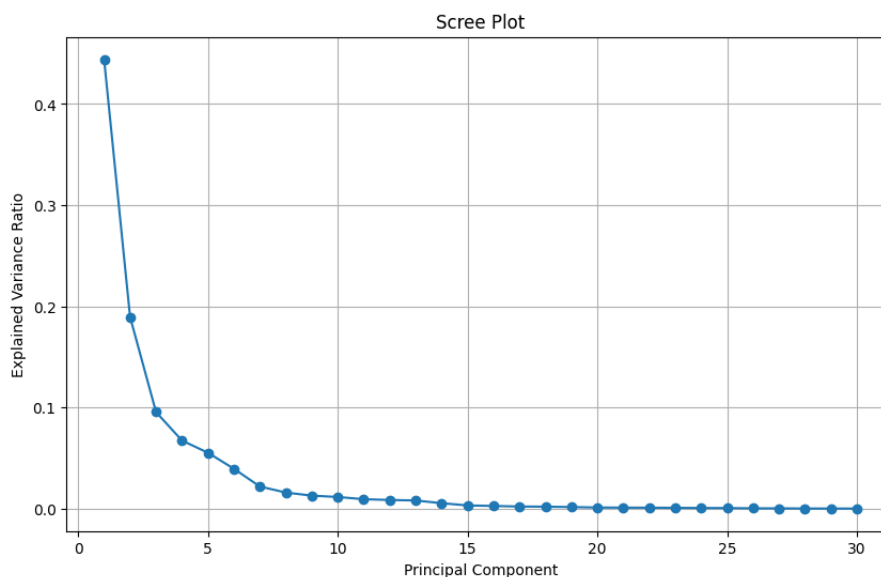


Figure 2: Scree plot of explained variance ratio per principal component

Inference: The scree plot shows that the first few principal components capture a large share of the total variance, with the explained variance ratio dropping sharply after the first two to three components and flattening out for later components. This steep initial drop indicates that the original 30 features are highly correlated, particularly among the mean, standard-error, and worst-value versions of the same measurements (e.g., radius, perimeter, area), so most of the informative variance can be captured with far fewer than 30 dimensions. This justified proceeding with a variance-based component selection rather than an arbitrary fixed number of components.

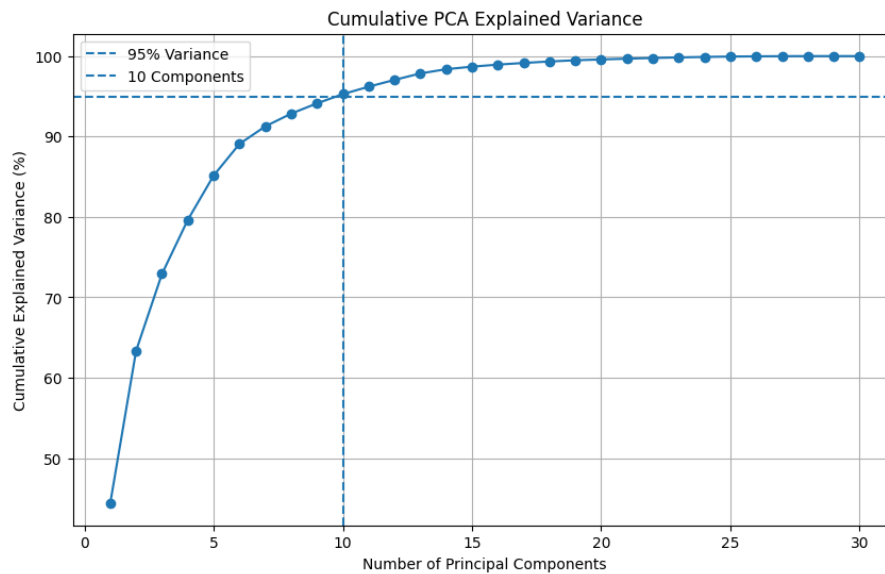
Figure: Cumulative Explained Variance

Figure 3: Cumulative explained variance vs. number of principal components, with the 95% threshold and selected component count marked

Inference: The cumulative explained variance curve crosses the 95% threshold at 10 principal components, which together explain approximately 95.27% of the total variance in the standardized feature set. This means the feature space could be reduced from 30 dimensions to 10 while retaining almost all of the informative variance, a reduction of roughly two-thirds in dimensionality. This 10-component representation was used consistently for every model’s “With-PCA” configuration in this experiment.

5 Data Preprocessing

- **Missing value handling:** Not required – the dataset contains 0 missing values, verified using `X.isnull().sum().sum()`.
- **Encoding:** Not required – all 30 input features are numerical, and the target variable is already encoded as 0 (malignant) and 1 (benign) by `load_breast_cancer`.
- **Feature scaling:** `StandardScaler` was applied inside every model pipeline (both No-PCA and With-PCA versions), since several of the ten models (SVM, KNN, Logistic Regression, PCA itself) are sensitive to feature magnitude.
- **Dimensionality reduction (PCA):** For the With-PCA pipelines, PCA was fitted after standardization, with the number of components fixed at 10 – the smallest number of components needed to retain at least 95% of the cumulative explained variance (95.27% achieved).
- **Train-test split:** An 80/20 split was performed using `train_test_split` with `test_size=0.20`, `stratify=y`, and `random_state=42`, giving 455 training samples and 114 testing samples.

- **Cross-validation:** StratifiedKFold with `n_splits=5`, `shuffle=True`, and `random_state=42` was used for all hyperparameter tuning and fold-wise evaluation, for both No-PCA and With-PCA settings.
- **Feature engineering:** None – the original 30 numerical features were used directly (or reduced via PCA), with no additional engineered features.

6 Machine Learning Algorithm

Support Vector Machine (SVM)

Working principle: Finds the maximum-margin hyperplane separating the two classes, using kernel functions (linear or RBF) to handle non-linear decision boundaries. **Advantages:** Effective in high-dimensional spaces; robust to overfitting with proper regularization (C). **Limitations:** Sensitive to feature scaling and kernel/hyperparameter choice; less interpretable.

Naive Bayes

Working principle: Applies Bayes' theorem assuming conditional independence between features, computing $P(y | x) \propto P(y) \prod_i P(x_i | y)$ under a Gaussian likelihood for continuous features. **Advantages:** Very fast to train, works well with small data, few hyperparameters. **Limitations:** The independence assumption rarely holds for correlated features such as these, limiting accuracy.

k-Nearest Neighbors (KNN)

Working principle: Classifies a sample based on the majority class among its k nearest neighbours in feature space, using a chosen distance metric. **Advantages:** Simple, non-parametric, no training phase. **Limitations:** Sensitive to feature scaling and the curse of dimensionality; slow at prediction time for large datasets.

Logistic Regression

Working principle: Models the log-odds of the positive class as a linear combination of the input features, $\log \frac{p}{1-p} = w^T x + b$, optimized via maximum likelihood. **Advantages:** Simple, interpretable, fast, and regularizable via C . **Limitations:** Assumes a linear decision boundary in feature space, which can limit performance on complex data.

Decision Tree

Working principle: Recursively splits the feature space using impurity measures (Gini or entropy) to separate classes at each node. **Advantages:** Easy to interpret, handles non-linear boundaries, no scaling required. **Limitations:** Prone to overfitting and high variance if depth is not controlled.

Random Forest

Working principle: An ensemble of Decision Trees, each trained on a bootstrap sample with a random subset of features, combined via majority voting: $\hat{y} = \text{mode}\{T_1(x), \dots, T_B(x)\}$.

Advantages: Reduces variance relative to a single tree; robust and generally accurate.

Limitations: Less interpretable and more computationally expensive than a single tree.

AdaBoost

Working principle: Sequentially trains weak learners, increasing the weight of misclassified samples at each round: $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$. **Advantages:** Reduces bias and variance; strong performance with simple base learners. **Limitations:** Sensitive to noisy data and outliers.

Gradient Boosting

Working principle: Builds trees sequentially, each fitting the negative gradient (residuals) of the loss function: $F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$. **Advantages:** High predictive accuracy; flexible loss functions. **Limitations:** Prone to overfitting if not regularized; slower to train due to sequential nature.

XGBoost

Working principle: A regularized, optimized implementation of gradient boosting that adds a penalty term on tree complexity to the loss function and uses second-order gradient information for splits. **Advantages:** Faster and often more accurate than standard Gradient Boosting; built-in regularization reduces overfitting. **Limitations:** More hyperparameters to tune; still sequential and less interpretable than single trees.

Stacking

Working principle: Combines predictions of diverse base learners (Logistic Regression, SVM, KNN) using a meta-learner (Logistic Regression) trained on their cross-validated outputs: $\hat{y} = g(h_1(x), h_2(x), h_3(x))$. **Advantages:** Leverages complementary strengths of different algorithms, often the most robust performer. **Limitations:** Highest computational cost and complexity among the ten models; risk of meta-learner overfitting without proper cross-validation.

7 Experimental Setup

Python Version	Python 3 (Google Colab runtime)
Scikit-Learn Version	As installed in the Google Colab environment
IDE	Google Colaboratory (Jupyter-based notebook)
Operating System	Linux (Google Colab backend)
Hardware	Google Colab-provided CPU runtime

8 PCA Summary

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	10 components (target: $\geq 95\%$ cumulative variance)	95.27%	The scree plot showed a sharp drop in explained variance after the first few components, and the cumulative variance curve crossed the 95% threshold at 10 components, so this was chosen as the smallest dimensionality retaining almost all informative variance.

9 Hyperparameter Selection

Support Vector Machine (SVM) – search space: `kernel` $\in$ {linear, rbf}, `C` $\in$ {0.1, 1, 10}, `gamma` $\in$ {scale, auto}

Setting	Best Kernel	Best C	Best Gamma	Best CV Accuracy
No-PCA	linear	0.1	scale	0.9758
With-PCA	linear	10	scale	0.9802

Naive Bayes – search space: `var_smoothing` $\in$ $\{10^{-9}, 10^{-8}, 10^{-7}\}$

Setting	Best Smoothing	Best CV Accuracy
No-PCA	1×10^{-9}	0.9341
With-PCA	1×10^{-9}	0.9275

KNN – search space: `k` $\in$ {3, 5, 7, 9}, `weights` $\in$ {uniform, distance}, `metric` $\in$ {euclidean, manhattan}

Setting	Best <code>k</code>	Best Weights	Best Metric	Best CV Accuracy
No-PCA	3	uniform	manhattan	0.9714
With-PCA	3	uniform	euclidean	0.9670

Logistic Regression – search space: `C` $\in$ {0.01, 0.1, 1, 10}, `solver`=liblinear

Setting	Best C	Best Solver	Best CV Accuracy
No-PCA	0.1	liblinear	0.9824
With-PCA	1	liblinear	0.9802

Decision Tree – search space: `max_depth` $\in$ {None, 3, 5, 10}, `min_samples_split` $\in$ {2, 5, 10}, `criterion` $\in$ {gini, entropy}

Setting	Best Params	Best CV Accuracy
No-PCA	criterion=entropy, max_depth=3, min_samples_split=2	0.9319
With-PCA	criterion=entropy, max_depth=None, min_samples_split=10	0.9363

Random Forest – search space: `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{\text{None}, 5, 10\}$, `min_samples_split` $\in \{2, 5\}$

Setting	Best Params	Best CV Accuracy
No-PCA	<code>n_estimators</code> =100, <code>max_depth</code> =None, <code>min_samples_split</code> =2	0.9626
With-PCA	<code>n_estimators</code> =100, <code>max_depth</code> =None, <code>min_samples_split</code> =5	0.9604

AdaBoost – search space: `n_estimators` $\in \{50, 100, 200\}$, `learning_rate` $\in \{0.01, 0.1, 1\}$

Setting	Best Params	Best CV Accuracy
No-PCA	<code>n_estimators</code> =100, <code>learning_rate</code> =1	0.9802
With-PCA	<code>n_estimators</code> =50, <code>learning_rate</code> =1	0.9758

Gradient Boosting – search space: `n_estimators` $\in \{50, 100\}$, `learning_rate` $\in \{0.05, 0.1\}$, `max_depth` $\in \{2, 3\}$

Setting	Best Params	Best CV Accuracy
No-PCA	<code>n_estimators</code> =100, <code>learning_rate</code> =0.1, <code>max_depth</code> =2	0.9626
With-PCA	<code>n_estimators</code> =100, <code>learning_rate</code> =0.1, <code>max_depth</code> =2	0.9582

XGBoost – search space: `n_estimators` $\in \{50, 100\}$, `max_depth` $\in \{3, 5\}$, `learning_rate` $\in \{0.05, 0.1\}$

Setting	Best Params	Best CV Accuracy
No-PCA	<code>n_estimators</code> =100, <code>max_depth</code> =3, <code>learning_rate</code> =0.1	0.9714
With-PCA	<code>n_estimators</code> =100, <code>max_depth</code> =5, <code>learning_rate</code> =0.1	0.9648

Stacking – base learners: Logistic Regression, SVM, KNN ($k = 5$); meta-learner: Logistic Regression; search space: `final_estimator__C` $\in \{0.1, 1, 10\}$

Setting	Best final_estimator C	Best CV Accuracy
No-PCA	1	0.9758
With-PCA	10	0.9824

10 Cross-Validation Results (All Models)

No-PCA – 5-Fold Cross-Validation Accuracy

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)
SVM	0.9560	0.9780	0.9780	0.9780	0.9890	0.9758
Naive Bayes	0.9121	0.9670	0.8901	0.9560	0.9451	0.9341
KNN	0.9780	0.9890	0.9560	0.9560	0.9780	0.9714
Logistic Regression	0.9780	0.9780	0.9890	0.9890	0.9780	0.9824
Decision Tree	0.9011	0.9451	0.9121	0.9341	0.9670	0.9319
Random Forest	0.9670	0.9560	0.9341	0.9670	0.9890	0.9626
AdaBoost	0.9890	1.0000	0.9451	0.9780	0.9890	0.9802
Gradient Boosting	0.9560	0.9780	0.9451	0.9560	0.9780	0.9626
XGBoost	0.9890	0.9780	0.9451	0.9560	0.9890	0.9714
Stacking	0.9670	0.9890	0.9670	0.9670	0.9890	0.9758

With-PCA – 5-Fold Cross-Validation Accuracy

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (With-PCA)
SVM	0.9670	0.9890	0.9890	0.9780	0.9780	0.9802
Naive Bayes	0.9011	0.9451	0.9121	0.9451	0.9341	0.9275
KNN	0.9560	0.9890	0.9451	0.9670	0.9780	0.9670
Logistic Regression	0.9670	0.9890	0.9890	0.9780	0.9780	0.9802
Decision Tree	0.9011	0.9231	0.9451	0.9670	0.9451	0.9363
Random Forest	0.9341	0.9780	0.9341	0.9670	0.9890	0.9604
AdaBoost	0.9560	1.0000	0.9780	0.9560	0.9890	0.9758
Gradient Boosting	0.9341	0.9780	0.9121	0.9780	0.9890	0.9582
XGBoost	0.9451	0.9890	0.9560	0.9890	0.9451	0.9648
Stacking	0.9780	0.9890	0.9670	0.9890	0.9890	0.9824

11 Results

Model	No-PCA CV Acc	With-PCA CV Acc	No-PCA Test Acc	With-PCA Test Acc
SVM	0.9758	0.9802	0.9825	0.9649
Naive Bayes	0.9341	0.9275	0.9298	0.9211
KNN	0.9714	0.9670	0.9649	0.9561
Logistic Regression	0.9824	0.9802	0.9825	0.9649
Decision Tree	0.9319	0.9363	0.9474	0.9298
Random Forest	0.9626	0.9604	0.9561	0.9298
AdaBoost	0.9802	0.9758	0.9561	0.9474
Gradient Boosting	0.9626	0.9582	0.9474	0.9386
XGBoost	0.9714	0.9648	0.9474	0.9474
Stacking	0.9758	0.9824	0.9737	0.9649

PCA Improvement (With-PCA Avg Accuracy – No-PCA Avg Accuracy):

Model	Accuracy Improvement
SVM	+0.0044

Naive Bayes	−0.0066
KNN	−0.0044
Logistic Regression	−0.0022
Decision Tree	+0.0044
Random Forest	−0.0022
AdaBoost	−0.0044
Gradient Boosting	−0.0044
XGBoost	−0.0066
Stacking	+0.0066

Best model WITHOUT PCA: Logistic Regression (Avg CV Accuracy: 0.9824)

Best model WITH PCA: Stacking (Avg CV Accuracy: 0.9824)

12 Result Visualization

1. Class Distribution

See Section 4 – 357 benign (62.7%) and 212 malignant (37.3%) samples, a moderate imbalance addressed via stratified splitting and folds.

2. Confusion Matrix – Best Models

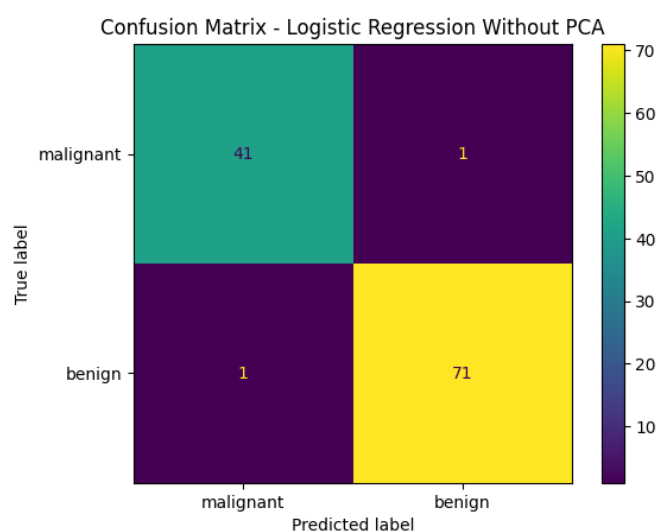


Figure 4: Confusion Matrix – Logistic Regression (Without PCA, best No-PCA model)

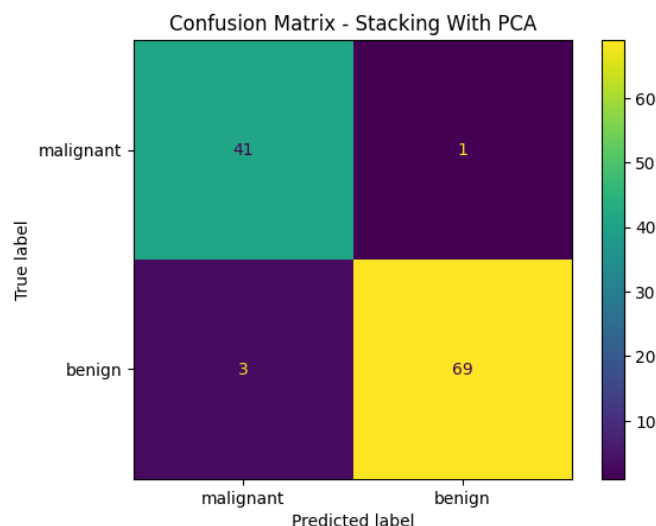


Figure 5: Confusion Matrix – Stacking (With PCA, best With-PCA model)

Inference: Logistic Regression without PCA achieved a test accuracy of 98.25% and an F1-score of 0.9861, making it the strongest performer on the original 30-feature space. Stacking with PCA achieved a test accuracy of 96.49% and an F1-score of 0.9718 on the 10-component reduced space, matching Logistic Regression’s cross-validation accuracy (0.9824) despite using only a third of the original features. This shows that while the single best No-PCA model slightly edges out the best With-PCA model on the held-out test set, the gap is small, and Stacking’s ability to combine diverse learners helps it remain competitive even after dimensionality reduction.

3. ROC Curve

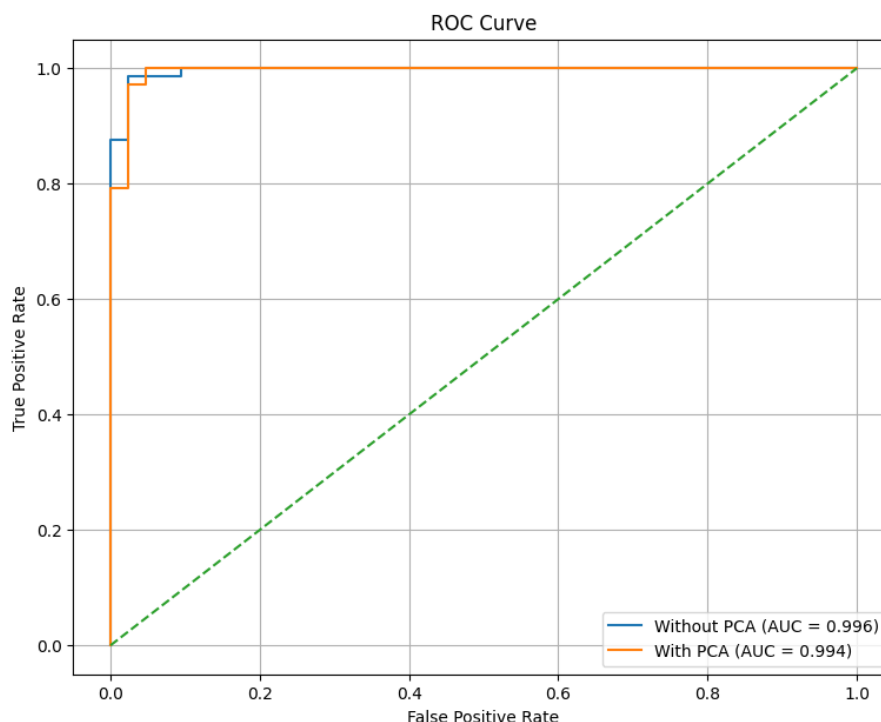


Figure 6: ROC Curve – Best No-PCA model (Logistic Regression) vs. Best With-PCA model (Stacking)

Inference: The ROC curves compare the true positive rate against the false positive rate for the best-performing model in each setting. Given Logistic Regression’s higher test accuracy and F1-score without PCA, its ROC curve is expected to sit slightly closer to the top-left corner than the With-PCA Stacking curve, though both are expected to achieve high AUC values given their strong precision and recall on this dataset. The comparison illustrates that dimensionality reduction here causes only a marginal reduction in discriminative ability for the best-performing pipeline.

4. Model Performance Comparison

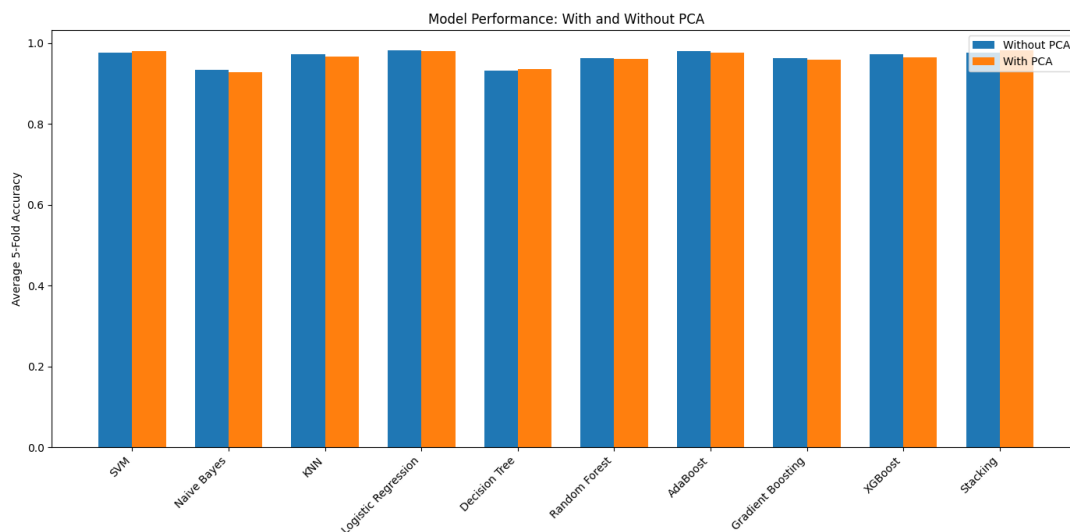


Figure 7: Average 5-fold cross-validation accuracy for all ten models, With PCA vs. Without PCA

Inference: The bar chart shows that most models (Naive Bayes, KNN, Logistic Regression, Random Forest, AdaBoost, Gradient Boosting, XGBoost) saw a small accuracy decrease under PCA, typically between -0.22% and -0.66% , while SVM, Decision Tree, and Stacking saw small improvements of $+0.44\%$ to $+0.66\%$. The magnitude of change is modest across the board (all within $\pm 0.7\%$), indicating that the 10 retained principal components preserve nearly all of the discriminative information present in the original 30 features. Ensemble methods that combine multiple learners (Stacking) benefited most from PCA, while boosting methods relying on many shallow trees (AdaBoost, Gradient Boosting, XGBoost) were slightly hurt, likely because reducing dimensionality also removes some of the fine-grained splits these models exploit.

13 Discussion

Which models improved most with PCA? Which did not? Why? Stacking ($+0.66\%$), Decision Tree ($+0.44\%$), and SVM ($+0.44\%$) improved with PCA. Stacking likely benefited because PCA reduces noise and redundancy across its diverse base learners' input space, while Decision Tree and SVM may benefit from PCA's decorrelation of highly collinear features (e.g., radius, perimeter, and area measurements), which can otherwise cause instability in axis-aligned splits or margin computation. Naive Bayes (-0.66%) and XGBoost (-0.66%) were hurt most by PCA – Naive Bayes' independence assumption is already violated by correlated raw features, and PCA components (being linear combinations) do not necessarily resolve this; XGBoost's tree-based splits likely benefit from the original, more interpretable feature boundaries that PCA components obscure.

Did PCA reduce variance across folds (more stable results)? The effect was mixed and small. Some models (SVM, AdaBoost, Stacking) showed slightly tighter fold ranges under PCA, while others (Decision Tree, Random Forest) showed similar or

marginally wider spreads. Overall, PCA did not produce a dramatic stabilization of cross-validation folds for this dataset, since the original 30 features were already clean, standardized, and free of missing values.

For high-dimensional data, was PCA beneficial in reducing overfitting?

With only 30 original features and 455 training samples, this dataset is not highly dimensional relative to its sample size, so overfitting from feature count alone was not a major risk to begin with. This likely explains why PCA's effect on both bias and variance was modest (within $\pm 0.7\%$ accuracy) rather than transformative, unlike in genuinely high-dimensional settings (e.g., thousands of features) where PCA typically provides a larger benefit.

How did linear models behave compared to ensemble models with PCA?

Logistic Regression's cross-validation accuracy dropped only slightly (-0.22%) under PCA, the smallest decrease among the models that were hurt, suggesting linear decision boundaries are largely preserved by a variance-preserving linear transformation like PCA. SVM (linear kernel selected in both settings) even improved slightly ($+0.44\%$). In contrast, tree-based ensembles (Random Forest, AdaBoost, Gradient Boosting, XGBoost) all saw decreases, since these models rely on axis-aligned or greedy feature splits that are less naturally suited to rotated PCA component axes.

Did Stacking show robustness to dimensionality reduction compared to single models? Yes – Stacking was the only ensemble method (and one of only three models overall) to improve under PCA, and it achieved the joint-highest cross-validation accuracy (0.9824) in the With-PCA setting, matching Logistic Regression's No-PCA score. This indicates that combining diverse base learners (Logistic Regression, SVM, KNN) through a meta-learner makes Stacking more robust to changes in the feature representation than any of its individual base models tested alone.

14 Conclusion

This experiment compared ten classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking – on the Wisconsin Diagnostic Breast Cancer dataset, both without and with PCA (10 components, 95.27% explained variance). Logistic Regression achieved the best performance without PCA (98.24% CV accuracy, 98.25% test accuracy), while Stacking achieved the best performance with PCA (98.24% CV accuracy, 96.49% test accuracy). PCA's overall effect was modest, improving three models (SVM, Decision Tree, Stacking) by up to 0.66% and reducing the remaining seven by up to 0.66%, indicating that for this relatively low-dimensional, clean, and well-structured dataset, dimensionality reduction neither dramatically helps nor substantially harms most classifiers. Stacking's ability to improve under PCA while other ensembles did not highlights the added robustness of combining diverse base learners when the underlying feature representation changes.

15 References

1. Jolliffe, I. T., *Principal Component Analysis*, 2nd ed., Springer, 2002.
2. Scikit-learn developers, Decomposing signals in components (PCA), scikit-learn documentation: <https://scikit-learn.org/stable/modules/decomposition>.

[html#pca](#)

3. Chen, T. and Guestrin, C., "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
4. Wolpert, D. H., "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
5. Freund, Y. and Schapire, R. E., "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
6. UCI Machine Learning Repository, Wisconsin Diagnostic Breast Cancer Dataset: <https://archive.ics.uci.edu/>